Historical Fragment Reuse and Behavior-Aware Position Ranking for Online Bin Packing

Abstract

Historical programs generated by large language models (LLMs) provide reusable components for automatic heuristic design. This project combines typed historical-fragment reuse with behavior-aware insertion-position ranking in genetic programming (GP) for online bin packing. A library of 1,240 eligible subtrees supports search with zero new LLM calls. For a fixed parent and vector fragment, the ranker compares up to three insertion positions using structural features and decisions on 64 fixed probe states, then fully evaluates one selected candidate. Across 30 paired seeds, historical-fragment GP uses 2.798 fewer test bins than basic GP and 2.939 fewer than a shape-matched random library. It improves development initialization in 12 of 30 runs, compared with 1 of 30 for basic GP and 2 of 30 for matched random-fragment GP. Frozen-ranker validation yields a 42.9% higher observed mean parent-retention gain with behavioral features than with structural features alone. In a separate 30-seed comparison with a 120-second budget per run, behavior-aware ranking achieves the lowest mean test bin count: 2086.016, versus 2087.889 for structural ranking and 2087.254 for random placement. These measured results demonstrate lower average packing costs from historical reuse and behavior-aware placement within the evaluated search budgets.

Keywords: automatic heuristic design; behavioral ranking; genetic programming; large language models; online bin packing; program reuse.

1 Problem and Motivation

Automatic heuristic design searches for executable decision rules instead of specifying every rule manually. Recent studies combine large language model (LLM) generation with evaluation and evolutionary selection [1–5]. These searches also accumulate programs whose subexpressions can remain useful after generation ends. Reusing this material offers a way to exploit historical LLM output without new model calls during a time-limited search. This project investigates that approach through typed fragment reuse in genetic programming (GP). Here, online search denotes the budgeted GP phase after offline archive preparation; online bin packing denotes sequential, irreversible item placement.

Online bin packing provides a concrete test of reuse. Items arrive sequentially, and each assignment is irreversible. A locally attractive placement can leave residual capacities that are difficult to fill later. Best-fit chooses the feasible bin with the smallest residual capacity after placement; its fixed rule does not adapt to the item distribution. GP searches for richer scoring functions, but many syntactically valid edits are ineffective, redundant, or harmful. Historical fragments supply an alternative to constructing every new subtree from basic primitives.

Effective reuse requires selecting both a fragment and its insertion context. Type compatibility makes an edit structurally admissible, but the surrounding operator and the displaced subtree determine its effect on the final scores. The same fragment can therefore help at one position and harm at another. The proposed ranker compares candidate sites using decisions on a small set of fixed probe states, then commits to one full fitness evaluation. This separates probe-based placement selection from the packing evaluations used to judge program quality.

Four research questions connect the design to its evaluation. RQ1 asks whether historical fragments provide more useful search material than basic primitives or shape-matched random fragments. RQ2 asks whether insertion position changes the effect of a fixed parent–fragment combination. RQ3 asks whether behavioral

features improve local selection beyond structural features. RQ4 asks whether the placement benefit persists in complete GP when probing and ranking are charged to a common time budget.

The project integrates typed historical reuse with context-dependent position ranking. It constructs candidates from archived subtrees, ranks insertion sites for a fixed parent and fragment, and evaluates reuse, local selection, and complete-search performance separately. Parent and fragment sampling remain unchanged; learning controls placement. The contribution is a complete pipeline that combines reusable historical program material, low-cost behavioral features, and budgeted GP, with paired comparisons against fragment and placement controls. LLM-generated code supplies the offline library, while online variation executes entirely on the CPU.

2 Literature Review

2.1 Scope and Organizing Perspective

This thematic review covers 20 papers [1–20] through their 2024–2026 publication or publicly released preprint versions. It groups research by the decision being automated: program generation, search control, evaluation allocation, and assessment of validity or transfer.

Wu et al. [1] distinguished LLM-enhanced evolutionary algorithms from evolutionary enhancement of LLMs. Liu et al. [2] organized LLM roles around design, optimization, prediction, and extraction. The present method belongs to LLM-enhanced evolution: it stores LLM-derived program material in a library and uses a lightweight ranker to control placement. The ranker itself is not an LLM, so online inference and historical generation have separate computational roles.

2.2 Generating and Evolving Executable Heuristics

FunSearch [3] couples LLM program proposals with an evaluator and a program population, including an online bin-packing demonstration. EoH [4] evolves natural-language heuristic ideas alongside executable code, while ReEvo [5] adds reflective verbal guidance. In each case, evaluation supplies task-specific feedback and search retains useful candidates. This evaluator-driven process motivates the present study: generated programs are not only final outputs but also a potential source of reusable components.

LLaMEA [6] applies iterative generation, mutation, and performance feedback to black-box metaheuristic design. Hemberg et al. [7] formalized LLM-based code evolution and examined how prompting shapes evolutionary operators. These approaches generally use a model during candidate variation. Typed subtree replacement instead restricts variation to explicit local edits that require no new model calls. The trade-off is reduced generative flexibility in exchange for a controlled search space; its effect on packing quality is evaluated independently of LLM usage.

AlphaEvolve [8] uses an evaluator-driven coding agent to improve algorithms across scientific and computational tasks. DeepEvolve [9] combines algorithm evolution with external research and debugging. Both emphasize validated executable changes and motivate preserving the computational knowledge produced during search. Their scope and infrastructure differ from the short-budget GP setting studied here, so they provide design context rather than numerical performance baselines.

2.3 Search Control and Experience Reuse

Promptbreeder [10] evolves task prompts and the prompts used to mutate them. EvoLLM [11] uses an LLM for in-context recombination in black-box optimization. These methods show that learning can target a search operator rather than only a final solution. The present method applies that perspective to insertion-site selection: a frozen ranker controls placement while parent and fragment sampling remain fixed.

HSEvo [12] combines diversity-oriented evolutionary heuristic design with harmony search. UBER [13] uses uncertainty to adjust exploration, exploitation, and island resets. ShinkaEvolve [14] combines parent

Table 1. Research Themes and Design Implications

Theme	Representative work	Design implication
Evaluated program generation	FunSearch, EoH, ReEvo, LLaMEA [3–6]	Reuse executable material; keep full fitness as the quality authority.
Learned search control	Promptbreeder, EvoLLM, HSEvo, UBER, ShinkaEvolve, EvoPH [10–15]	Specify exactly which search decision the ranker controls.
Cost and coverage	MEoH and EoH-S [16, 17]	Include execution overhead and evaluate coverage across input settings.
Experimental validity	Benchmarking, LLM4AD, synthesis comparison [18–20]	Use explicit contracts, meaningful controls, and repeated paired runs.

sampling, code novelty rejection, and bandit-based model selection. These mechanisms control different stages of program search. Position ranking acts after the parent and fragment have been selected, isolating a conditional variation decision rather than changing the broader search policy.

EvoPH [15] co-evolves prompts and heuristics through experience and population mechanisms. The present method also reuses history, but represents it as typed executable subtrees and selects placements using numerical structural and behavioral features. The research question is therefore not whether experience can be reused in general, but whether archived subtree content improves bounded GP search and whether conditional placement ranking adds value.

2.4 Cost, Diversity, and Generalization

MEoH [16] treats heuristic quality and practical criteria as a multi-objective design problem, motivating joint reporting of computation and packing quality. This study instead imposes a fixed wall-clock budget. Charging proposal construction, probing, ranking, and full evaluation avoids favoring a selector merely because its overhead is omitted from an evaluation-count comparison.

EoH-S [17] searches for complementary heuristic sets for heterogeneous instances. Its formulation highlights the limits of relying on a single policy across different inputs. The present system returns one best program and evaluates it on a specific test family. Retaining complementary programs or fragments is a natural extension, but its coverage and selection cost require separate evaluation.

2.5 Experimental Validity and Research Gap

Zhang et al. [18] identified inconsistent settings, weak baselines, and insufficient component analysis in LLM-based heuristic-search benchmarks. LLM4AD [19] provides modular search and evaluation infrastructure. Sobania et al. [20] compared LLM and GP program synthesis, reporting differences in structure, diversity, and repeatability and discussing benchmark exposure. These concerns motivate the present study's conventional GP controls, matched random fragments, paired seeds, held-out parent groups, and separation of development fitness from final test results.

Table 1 connects the reviewed research themes to the design requirements of executable reuse, explicit search control, cost-aware evaluation, and experimental validity. The method combines modular program reuse, behavior-informed variation, and learned evolutionary control. Its design uses a fixed historical subtree library, up to three positions for each eligible parent-fragment pair, a frozen structural-plus-behavioral ranker, and one complete evaluation of the selected candidate. Probe decisions supply the behavioral features used for placement. Full packing evaluation supplies the fitness used for population retention. The experiments measure the resulting packing quality with online costs included.

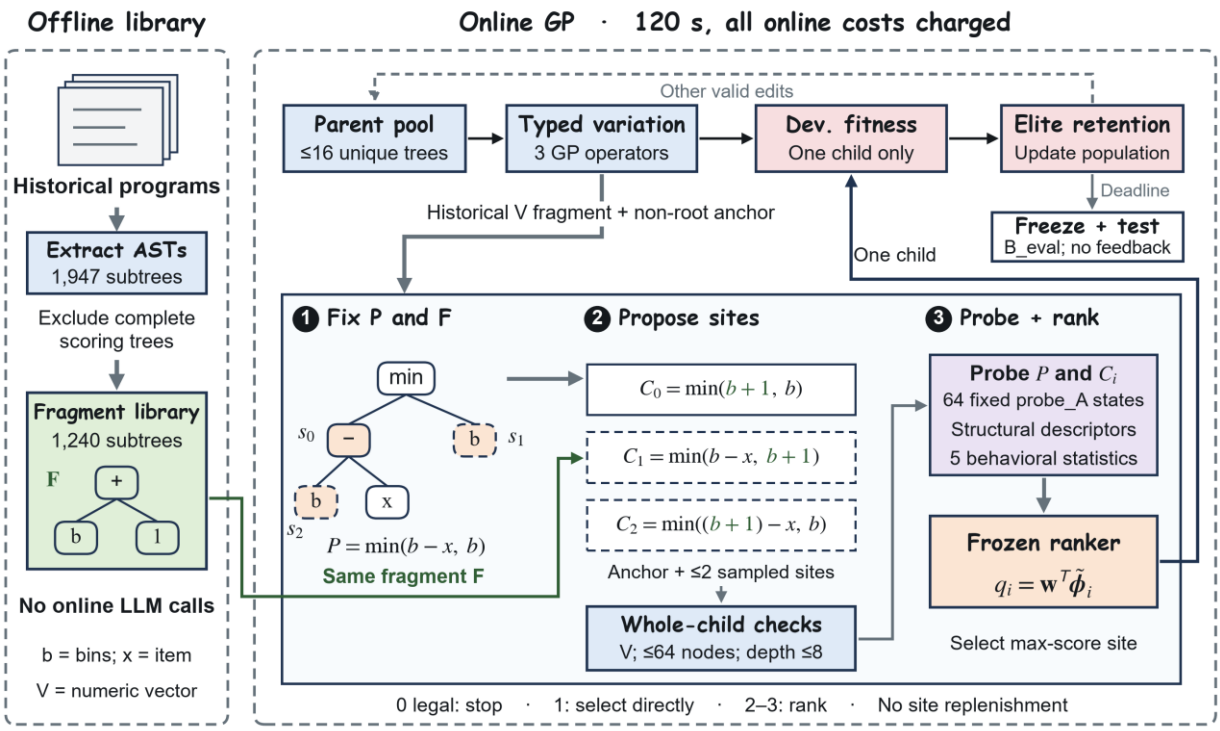


Figure 1. Historical-Fragment GP with Behavior-Aware Position Ranking. The upper loop shows the 120-second experimental configuration; the inset illustrates conditional placement. Orange nodes mark insertion sites, with s_0 as the anchor; dashed nodes and candidate boxes denote optional sampled extras. Green terms identify the same inserted fragment F, not a preferred candidate. Here min is elementwise minimum, and the expressions are illustrative rather than measured search outcomes. For two or three legal candidates, the frozen ranker combines structural descriptors with five behavioral statistics from 64 probe_A states. The tilde denotes feature-wise division by frozen training-set root-mean-square scales; q_i is the ranking score. Only the selected child receives full development evaluation; final B_eval testing gives no search feedback.

3 Methodology

Figure 1 summarizes the search pipeline and illustrates how a fixed historical fragment is compared across insertion positions.

3.1 Decision Interface and Objective

Each item has size x, and each bin has capacity C. At each decision, the policy receives x and a vector r of residual capacities for existing feasible bins, where r_j ≥ x for each feasible bin j. It returns score(x, r), a finite real-valued vector of the same length, without modifying its inputs. The highest-scoring feasible bin is chosen; ties go to the earliest-opened bin. A new bin is opened only when no existing bin is feasible. Future items are unavailable to the policy.

For a program P, let J_D(P) be its mean final number of bins across development instances D; smaller is better. Given fragment F and replacement site s, define C_s = P[F @ s] and Δ_s = J_D(P) − J_D(C_s). A positive Δ_s denotes improvement. If complete execution fails, Δ_s is undefined rather than an observed zero. For a separate parent-retention metric, define G_s = max(0, Δ_s) for valid candidates and G_s = 0 for failures, because retaining the parent then gives no improvement. Validity must be reported separately so that this convention does not conceal failures.

3.2 Historical Fragments and Evolutionary Search

The method uses typed, tree-based genetic programming (GP) to evolve bin-scoring functions from reusable historical code. Each function is represented as an abstract syntax tree (AST), and its subtrees serve as candidate fragments. Historical functions are converted into self-contained scoring expressions before subtree extraction and structural deduplication. From 1,947 distinct fragments, the method removes 707 that match normalized complete historical scoring trees, leaving 1,240 eligible fragments. This allows the search to reuse components rather than insert complete archived strategies. Type compatibility ensures that a fragment replaces only an expression with the same output type.

Each run initializes the population with Best-fit and 15 randomly generated expressions. Search uses three operators: typed subtree crossover between parents, subtree replacement, and Gaussian mutation of numeric constants. For subtree replacement, the method samples a compatible historical fragment with probability 0.5; otherwise, it generates a new subtree from basic primitives. Historical fragments are sampled uniformly within the required type, without weighting by their archive frequency or previous fitness. Retained offspring can become parents, allowing useful fragments to be recombined over successive search steps.

A complete candidate must return a numeric vector and satisfy limits of 64 expression nodes and depth 8. After the insertion-site selection described in Section 3.3, only the selected candidate receives full development evaluation. Elitist selection retains up to 16 distinct valid programs, preferring fewer development bins and then smaller trees, with a deterministic final tie-break. The historical-reuse experiment uses 112 offspring attempts, while the placement experiment repeats the same operator schedule within its time budget. Position ranking changes where an eligible historical fragment is inserted, not the other GP operators or the population-selection rule.

3.3 Conditional Position Proposals

Each proposal begins by fixing the operation and sampling a receiving parent. The implementation traverses that parent's expression operands, recording each node's type and its path from the root. A path consists of AST field names and, where needed, argument-list indices; function names and indexing metadata are not replacement sites. An anchor site is sampled uniformly from these locations, and its output type determines the compatible donor pool. For a historical subtree-replacement operation, one fragment is then sampled uniformly from the eligible library entries of that type. The parent and fragment remain fixed while the proposal compares insertion positions.

Multiple-position selection is enabled only when the sampled replacement source is historical and the anchor is a non-root V-type site. Here, non-root refers to position in the parent tree, not to whether the selected subtree has children: a vector terminal such as bins is also eligible when it occurs below the root. The original anchor is always included. The procedure lists the other non-root V-type paths, sorts their serialized paths to obtain a reproducible ordering, and samples up to two without replacement. Consequently, a proposal contains the anchor plus zero, one, or two additional positions. Crossover, basic subtree generation, scalar or mask replacement, constant mutation, and historical replacement at the root retain a single-position proposal.

For every proposed path s, the builder creates a separate child C_s = P[F @ s] by replacing that subtree with a deep copy of the same fragment F. It records the site depth, displaced-subtree size, enclosing operator, and complete-child size and depth. The complete-child type and the 64-node and depth-8 limits are checked after construction. Positions are sampled before these checks, and rejected positions are not replenished from the remaining tree. A proposal with no structurally legal child ends the attempt; one legal child is selected directly; two or more legal children create a ranking opportunity. Distinct paths remain separate proposal slots even when they produce the same child AST.

For illustration, consider the parent P = np.minimum(bins - item, bins) and the eligible library fragment F = bins + 1. Replacing the first argument produces np.minimum(bins + 1, bins), whereas replacing the second produces np.minimum(bins - item, bins + 1). The bins terminal inside the first argument is another non-root

Table 2. Placement Selectors and Evaluation Requirements

Selector	Selection information	Full evaluations per proposal
Random position	Uniform choice among the common legal sites	One selected candidate
Structural ranking	Frozen model using structure only	One selected candidate
Behavior A ranking	Frozen model using structure and probe_A behavior	One selected candidate
Post hoc oracle	Complete quality of every proposed site	All sites; diagnostic only

V-type site; replacing it produces np.minimum((bins + 1) - item, bins). The three edits use identical fragment content but preserve different surrounding computations. This example illustrates tree replacement, not a measured quality improvement.

The selectors in Table 2 differ only after the common proposal and structural checks. Random placement samples uniformly among the surviving positions. Structural ranking computes the frozen score from structural features. Behavior A additionally evaluates the parent and each legal child on the same 64 probe_A states before scoring the positions. Learned selectors choose the highest-scoring site, resolving ties with the fixed hash rule in Section 3.5. Only the chosen child receives full development fitness evaluation and can enter the population. Neither ranking nor complete candidate fitness influences the initial site sample, and there is no fitness-guided resampling. Random-number streams are keyed by search seed, attempt, and decision channel rather than selector identity; later parent populations can still diverge after different selections.

3.4 Structural and Behavioral Features

Structural features encode insertion depth, replaced-subtree node count, resulting program size and depth, and the operator above the site. Behavioral features encode probe validity, the proportion of states with comparable parent and candidate decisions, the proportion with all scores tied, and the fraction of comparable decisions that change, together with its square. Operator encoding, denominators, missing-value rules, scaling, and score tie-breaking belong to the frozen model configuration; they are not re-estimated from test batches.

The probes comprise 128 fixed states, divided equally into probe_A and probe_B. They come from development trajectories of Best-fit, EoH, and Historical-best and cover different packing stages and numbers of feasible bins. The primary behavior-aware selector, Behavior A, uses probe_A for training and deployment observations. The probe_B set supports sensitivity analysis with fixed coefficients. Both sets are separate from the final packing test set, B_eval. Parent and candidate decisions are compared on the same fixed states rather than on separate evolving trajectories.

Behavioral observations describe how an edit changes decisions at a proposed site. Probe validity, tied scores, and changed decisions provide complementary inputs to the ranker. A changed decision can affect future residual capacities, so the ranker combines these observations with the surrounding program structure. Complete packing evaluation determines the fitness used for population retention.

3.5 Training and Deployment

The structural and behavior-aware selectors use pairwise linear ridge rankers with fixed regularization lambda = 1. Training comparisons are formed within parent-fragment groups. Among valid candidates, fewer bins is preferred; a valid candidate outranks an execution failure; equal-quality candidates and pairs of failures are ties. Each group receives equal total weight. Grouped cross-prediction holds out all fragments and positions associated with a source parent or seed together, keeping each held-out group intact during training and validation.

The ranking score is $r_s = w^T \varphi_s$, where φ_s is the feature vector for site s and w is the learned weight vector. The linear model has no intercept. Unordered within-group position pairs receive labels +1, -1, or 0, including tied pairs, and equal shares of their group's total weight. Feature differences are scaled by their weighted root-mean-square values; scales below 10^{-12} are set to one. Coefficients, scaling, and feature order are frozen before new-parent validation and online search. Score ties use a fixed hash of the salt, parent AST, fragment key, and target path.

During deployment, behavioral information determines the order of compatible insertion positions. The selector fully evaluates the chosen child, and development fitness determines population retention. This separates the low-cost placement decision from the complete packing objective while preserving one full evaluation per selected proposal.

3.6 Budgeted Online Procedure

1. Start the online clock, load the required archive and model, and construct the seed-specific initial population.

2. Until the deadline, select an operation and parent under the frozen search configuration. Generate one ordinary candidate unless the operation is eligible for position ranking.

3. For an eligible operation, fix the fragment, propose at most three sites, and apply common type and size checks. If no legal site remains, record the attempt and continue. Otherwise, select the sole legal site or rank the multiple legal sites.

4. Fully evaluate only the selected candidate on development data. Record failures, duplicates, proposal features, ranking scores, and all costs.

5. Update the population only if evaluation and submission satisfy the deadline rule. Charge late work but do not allow a late result to update the population.

6. Freeze the best program submitted within the deadline and evaluate it on B_eval after search ends. Keep B_eval results separate from search and ranker updates.

Parent probe outputs use a per-run cache keyed by the execution AST, state-pool hash, and state identifier. Each run begins with a cold cache, and cache construction, queries, and misses are charged to the online budget. Hit rates are recorded separately. Offline program generation, archive construction, label acquisition, and model training have separate cost records. The GP search stage makes zero new LLM calls.

4 Experimental Setup

4.1 Data and Experimental Units

Development fitness uses five pilot_dev instances, each containing 5,000 items with bin capacity 100. Item-size preprocessing uses floor rounding in pilot_dev and round in the subsequent B test configuration.

The historical-reuse and complete-GP placement experiments each use 30 paired search seeds per method, but their B test datasets differ. Their absolute bin counts are therefore analyzed separately. In the placement study, each frozen program is evaluated on the same 30 B_eval instances. Test bins are averaged first over instances within each seed, then over the 30 seeds, giving equal seed weights. Local diagnostics use parent–fragment groups rather than search seeds as their comparison units.

Table 3. Experimental Settings and Comparison Units

Study	Comparison or unit	Budget and role
Historical reuse	Basic GP, historical-fragment GP, fixed-parent sampling, matched random-fragment GP; 30 paired seeds	112 offspring attempts in the fixed-attempt setting
Local placement	Fixed parent–fragment groups with multiple sites	One full evaluation as primary selection budget; all sites for diagnosis
Complete GP placement	Random placement, structural ranking, Behavior A; 30 paired seeds each	120 seconds online per run; 90 runs total
Final placement test	30 common B_eval instances per frozen program	Post-search evaluation; no model or population updates

4.2 Controls and Ablations

Basic GP removes historical fragments, while historical-fragment GP retains them in an evolving population. The matched random-fragment control matches output type, node count, and depth to distinguish historical content from these structural properties. A fixed-parent sampling ablation restricts receiving and supplying parents to the initialization pool, targeting the value of population updates.

Behavior A versus structural ranking is the primary placement comparison: both share the proposal mechanism, but only Behavior A uses behavioral inputs. Random placement provides a second control within historical GP and is distinct from basic GP. The full-information oracle evaluates every proposed site.

4.3 Time Accounting and Reproducibility

Each placement run has a nominal 120-second online budget: 3,600 seconds per method and 10,800 seconds across 90 runs. Chargeable work includes loading, initialization, tentative replacements, probes, ranking, complete fitness evaluation, and required logging. The deadline rule permits population updates only from results submitted on time. Last legal submission, scheduler termination, and cleanup completion are distinct events; cleanup after the deadline is not itself additional search.

The archived runs use an Intel Core i7-12700H CPU with 20 logical processors, Windows 11, Python 3.12.14, and NumPy 2.3.5. One search run and one candidate worker execute at a time, with one BLAS thread and a 512 MiB candidate memory limit. Operation schedules repeat shuffled blocks of 56 crossovers, 39 subtree replacements, and 17 constant mutations, stopping at the run deadline.

4.4 Metrics and Statistical Analysis

Packing quality is measured by bin count, with lower values preferred. Complete-search results report the mean and sample standard deviation of seed-level test means, paired differences, 95% intervals, wins, ties, losses, completed offspring, and sortable proposals. For seed k, define d_k = mean_test_bins(control,k) - mean_test_bins(Behavior A,k); positive values favor Behavior A. Bootstrap resampling uses paired search seeds after averaging the test instances within each seed.

The archived combined-GP analysis uses 10,000 paired percentile bootstrap draws with random seed 2026101812 and the 2.5th and 97.5th percentiles. Seed differences are computed from the saved per-instance bin counts. A crossed seed-and-instance bootstrap is also supplied in the comparison file. Displayed complete-GP quality results use three decimal places.

Local validation reports mean parent-retention gain, assigning zero to failed or non-improving edits because the parent is retained. Signed gain is recorded for valid executions as defined in Section 3.1. Each parent-

Table 4. Historical-Reuse Results across 30 Paired Seeds. Mean test bins are reported on the reuse study's B data; lower is better.

Method	Search seeds	Mean bins
Basic GP	30	2092.312
Historical-fragment GP	30	2089.514
Fixed-parent fragment sampling	30	2090.507
Matched random-fragment GP	30	2092.453

Table 5. Frozen-Ranker Validation on New Parent–Fragment Pairs. Mean parent-retention gains and their difference are in bins; positive differences favor Behavior A.

Quantity	Reported value
Behavior A mean parent-retention gain	0.254237
Structural ranking mean parent-retention gain	0.177966
Behavior A minus structural ranking	0.076271
Descriptive 95% interval for the difference	[−0.072222, 0.366957]

fragment group is the comparison unit, and the archived results retain the selected positions, complete evaluations, and validity records.

5 Results and Analysis

5.1 Value of Historical Fragments

Historical-fragment GP achieves the lowest mean bin count in the reuse experiment (Table 4): 2089.514, versus 2092.312 for basic GP, 2090.507 for fixed-parent fragment sampling, and 2092.453 for matched random-fragment GP. The respective average reductions are 2.798, 0.992, and 2.939 bins. The matched-library comparison has a descriptive 95% interval of [1.242, 4.846]. Historical-fragment GP also improves development initialization in 12 of 30 runs (40.0%), compared with 1 of 30 for basic GP and 2 of 30 for matched random-fragment GP. These results connect the historical library to both lower average test bin counts and more successful improvements from the common starting population.

5.2 Insertion Context and Local Ranking

Insertion position can reverse a fragment's effect. Among 124 parent–fragment groups in the competitive-parent diagnostic with at least two completely valid positions, 19 contain both an improving and a worsening site: 15.3% of this subset. Because the parent and fragment remain fixed within each group, these sign reversals directly motivate selecting insertion context rather than treating type-compatible sites as interchangeable.

Behavior A achieves a mean parent-retention gain of 0.254237 bins on new parent-fragment combinations, versus 0.177966 for structural ranking (Table 5). The increase of 0.076271 bins corresponds to a 42.9% higher observed mean gain. This percentage describes the local improvement delivered by selected edits with parent retention. The comparison fixes the parent and fragment within each group, directly measuring the average gain from choosing an insertion position with behavioral features.

Table 6. Complete-GP Placement Results. Each method uses 30 paired seeds and a 120-second budget per run. SD is the sample standard deviation of seed-level mean test bins.

Method	Mean bins	Seed SD	Mean completed offspring	Sortable proposals total
Random position	2087.254	6.421	108.70	219
Structural ranking	2087.889	4.960	112.00	237
Behavior A ranking	2086.016	7.249	111.77	237

Table 7. Paired Improvements for Behavior A. Positive bin reductions and wins favor Behavior A; intervals use the archived paired-seed bootstrap.

Control	Mean improvement	Paired 95% interval	Win / tie / loss
Structural ranking	+1.873 bins	[−0.619, 4.593]	8 / 17 / 5
Random position	+1.239 bins	[−1.840, 4.492]	8 / 18 / 4

5.3 Complete GP under a Common Time Budget

Behavior A achieves the lowest mean test bin count under the 120-second budget (Table 6). Its mean of 2086.016 improves on structural ranking by 1.873 bins and random placement by 1.239 bins in the paired analysis (Table 7), approximately 0.090% and 0.059% of the respective control means. Differences are computed before rounding the table entries. Behavior A records 8 wins and 5 losses against structural ranking and 8 wins and 4 losses against random placement. Its observed average advantage extends to both placement controls.

The comparisons cover all 30 paired seeds, including 17 ties against structural ranking and 18 ties against random placement. Table 7 reports the paired differences and intervals, while Table 6 reports the complete seed-level variation. All seeds contribute equally to the reported mean packing quality.

Behavior A completes 111.77 offspring per run on average, close to structural ranking's 112.00 and above random placement's 108.70. Its lower mean bin count therefore accompanies similar observed completion counts to structural ranking. The methods encounter 219 to 237 sortable proposals over 30 runs, equivalent to 7.30 to 7.90 opportunities per run. Probe execution, ranking, and full evaluation are included in the shared 120-second budget.

5.4 Fixed References and Research Questions

Behavior A uses 4.484 fewer bins on average than Best-fit on the placement test data (Table 8). The fixed EoH and Historical-best programs provide additional quality references with their recorded mean bin counts. The search starts from Best-fit and basic random expressions and constructs candidates through fragment recombination. Complete historical scoring trees are excluded from direct fragment insertion.

The results answer the research questions at distinct levels. For RQ1, historical-fragment GP achieves lower mean bin counts than basic GP, fixed-parent sampling, and matched random-fragment GP. For RQ2, changing insertion context reverses the sign of an edit in 19 diagnostic groups. For RQ3, behavioral features yield a 42.9% higher observed mean local parent-retention gain. For RQ4, Behavior A achieves the lowest mean complete-search bin count while completing nearly as many offspring as structural ranking. The reuse and placement studies are reported separately using their respective test datasets.

Table 8. Fixed-Policy Quality References on B_eval. Mean test bins are directly comparable with Table 6; lower is better. These policies are not 30-run search methods.

Policy	Mean bins	Interpretation
Best-fit	2090.500	Conventional fixed heuristic
EoH reference	2083.633	Fixed externally generated reference policy
Historical-best	2067.300	Fixed historical quality reference

6 Discussion

6.1 Design Implications

Historical reuse separates the acquisition of program material from its online recombination. Once the archive exists, GP can exploit its typed fragments without invoking an LLM during variation. The matched-library result supports the value of that material in the tested setting. Position ranking adds a focused decision layer: it compares candidate contexts using probes while reserving complete fitness evaluation for one selected edit. This preserves a clear separation between placement prediction and the evaluation used for population retention.

The local and complete-search evaluations favor Behavior A in their observed average results. Local validation holds the parent and fragment fixed, while complete GP allows selected offspring to become later parents. The saved proposal and ancestry traces connect insertion choices with population updates and final candidates. Reporting these two levels shows both the local placement gain and the final packing quality obtained within the run budget.

6.2 Evaluation and Reproducibility Controls

The reuse comparison holds initialization, candidate budgets, type checks, and size limits constant across methods. The random fragment library matches output type, node count, and depth, and fixed-parent sampling isolates the role of population updates. The placement comparison holds the ranking family, shared site proposals, and time budget constant while adding behavioral features. Each method's realized proposals and population transitions are saved for inspection.

Probe states are fixed before placement evaluation and drawn from development trajectories of Best-fit, EoH, and Historical-best. The collection covers early, middle, and late packing stages and different feasible-bin counts. Parent and candidate decisions are measured on the same states, and the frozen coefficients are shared between the primary probe_A analysis and the probe_B sensitivity analysis.

Each development and placement-test instance contains 5,000 ordered items with capacity 100. The saved generator settings use a Weibull distribution with shape 3 and scale 45, clipped to [1,100]. Development data use floor rounding; the placement test data use the official round convention. Instance identifiers, random seeds, item-sequence hashes, and nonoverlap checks accompany the data, making the preprocessing and evaluation order reproducible.

Statistical reporting retains every paired seed and its instance results. Means weight the search seeds equally, and paired differences preserve the relationship between methods on the same seed. The released summaries include mean bin counts, standard deviations, win/tie/loss counts, and bootstrap intervals. The full result tables contain 120 historical-study runs and 90 position-study runs.

6.3 Cost and Practical Value

The online method requires zero new LLM queries and charges archive loading, proposals, probes, ranking, and full evaluation to the search budget. Historical generation and offline preparation are recorded separately. Let C_online denote one online search cost and C_off the offline preparation cost. Reusing the fixed archive and model over M deployments allocates an average preparation-plus-search cost of C_online + C_off/M. The implementation supports this reuse by retaining the typed fragment library, frozen models, and fixed probe states across searches, while resetting each run's online cache.

6.4 Further Design Directions

Further design directions include multiple independently generated fragment libraries, richer primitive-frequency matching, additional item distributions, and longer search budgets. These extensions build on the current controls, recorded seed lists, fixed stopping rules, and complete instance-level outputs.

The saved mechanism records cover ranking eligibility, selector disagreements, candidate validity, complete evaluations, retained gains, offspring ancestry, and timing. They support analysis of how local position choices propagate through GP. Additional directions include broader probe coverage, larger candidate-site sets, and alternative rankers, with their execution costs charged to the same total-time accounting procedure.

7 Conclusion

Historical LLM-generated fragments provide useful material for typed GP in the evaluated online bin-packing setting. With zero new online LLM calls, historical-fragment GP reduces mean test bins by 2.798 relative to basic GP and 2.939 relative to a shape-matched random library. It improves the development starting point in 40.0% of runs. Position diagnostics also identify parent-fragment groups in which changing the insertion site changes an edit from an improvement to a regression, motivating placement-aware reuse.

Behavior-aware ranking compares up to three positions using 64 fixed probe states and fully evaluates one selected candidate. It achieves a 42.9% higher observed mean local parent-retention gain than structural ranking. In the separate 120-second GP comparison, it achieves the lowest mean bin count: 1.873 fewer bins than structural ranking and 1.239 fewer than random placement, with probe and ranking costs included. These measured advantages combine reusable historical program material with behavior-guided placement in a CPU-only search pipeline.

References

[1] X. Wu, S.-h. Wu, J. Wu, L. Feng, and K. C. Tan. Evolutionary Computation in the Era of Large Language Model: Survey and Roadmap. IEEE Transactions on Evolutionary Computation, 29(2), 534–554, 2025. https://arxiv.org/abs/2401.10034

[2] F. Liu et al. A Systematic Survey on Large Language Models for Algorithm Design. ACM Computing Surveys, 2026. https://doi.org/10.1145/3787585

[3] B. Romera-Paredes et al. Mathematical discoveries from program search with large language models. Nature, 625, 468–475, 2024. First online December 2023. https://doi.org/10.1038/s41586-023-06924-6

[4] F. Liu, X. Tong, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, and Q. Zhang. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. ICML, 2024. https://arxiv.org/abs/2401.02051

[5] H. Ye et al. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. NeurIPS, 2024. https://arxiv.org/abs/2402.01145

[6] N. van Stein and T. Bäck. LLaMEA: A Large Language Model Evolutionary Algorithm for Automatically Generating Metaheuristics. IEEE Transactions on Evolutionary Computation, 2025. https://doi.org/10.1109/TEVC.2024.3497793

[7] E. Hemberg, S. Moskal, and U.-M. O'Reilly. Evolving code with a large language model. Genetic Programming and Evolvable Machines, 25, article 21, 2024. https://doi.org/10.1007/s10710-024-09494-2

[8] A. Novikov et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. White paper, arXiv:2506.13131, 2025. https://arxiv.org/abs/2506.13131

[9] G. Liu, Y. Zhu, J. Chen, and M. Jiang. Scientific Algorithm Discovery by Augmenting AlphaEvolve with Deep Research. Preprint, arXiv:2510.06056, 2025. https://arxiv.org/abs/2510.06056

[10] C. Fernando, D. S. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. ICML, PMLR 235, 13481–13544, 2024. https://proceedings.mlr.press/v235/fernando24a.html

[11] R. T. Lange, Y. Tian, and Y. Tang. Large Language Models As Evolution Strategies. Preprint, arXiv:2402.18381, 2024. https://arxiv.org/abs/2402.18381

[12] P. V. T. Dat, L. Doan, and H. T. T. Binh. HSEvo: Elevating Automatic Heuristic Design with Diversity-Driven Harmony Search and Genetic Algorithm Using LLMs. Preprint, arXiv:2412.14995, 2024. https://arxiv.org/abs/2412.14995

[13] Z. Chen, Z. Zhou, Y. Lu, R. Xu, L. Pan, and Z. Lan. UBER: Uncertainty-Based Evolution with Large Language Models for Automatic Heuristic Design. Preprint, arXiv:2412.20694, 2024. https://arxiv.org/abs/2412.20694

[14] R. T. Lange, Y. Imajuku, and E. Cetin. ShinkaEvolve: Towards Open-Ended And Sample-Efficient Program Evolution. Preprint, arXiv:2509.19349, 2025. https://arxiv.org/abs/2509.19349

[15] Y. Liu, J. Li, W. X. Zhao, H. Lu, and J.-R. Wen. Experience-Guided Reflective Co-Evolution of Prompts and Heuristics for Automatic Algorithm Design. Preprint, arXiv:2509.24509, 2025. https://arxiv.org/abs/2509.24509

[16] S. Yao, F. Liu, X. Lin, Z. Lu, Z. Wang, and Q. Zhang. Multi-objective Evolution of Heuristic Using Large Language Model. AAAI, 2025. https://arxiv.org/abs/2409.16867

[17] F. Liu, Y. Liu, Q. Zhang, X. Tong, and M. Yuan. EoH-S: Evolution of Heuristic Set using LLMs for Automated Heuristic Design. Preprint, arXiv:2508.03082, 2025. https://arxiv.org/abs/2508.03082

[18] R. Zhang, F. Liu, X. Lin, Z. Wang, Z. Lu, and Q. Zhang. Understanding the Importance of Evolutionary Search in Automated Heuristic Design with Large Language Models. PPSN, 2024. https://arxiv.org/abs/2407.10873

[19] F. Liu et al. LLM4AD: A Platform for Algorithm Design with Large Language Model. Preprint, arXiv:2412.17287, 2024. https://arxiv.org/abs/2412.17287

[20] D. Sobania, J. Petke, M. Briesch, and F. Rothlauf. A Comparison of Large Language Models and Genetic Programming for Program Synthesis. IEEE Transactions on Evolutionary Computation, author accepted manuscript, 2024. https://discovery.ucl.ac.uk/id/eprint/10192702/

Appendix A Evidence and Reproduction Materials

The method and results are linked to the archived experiment records and the repository at https://github.com/robert06hw/dsai. The historical study contains four methods and 30 paired seeds. The position study combines the initial 10 seeds and the first 20-seed extension, with three methods per seed. Tables 4 to 8 present the recorded results and arithmetic comparisons. The literature references describe the related research separately from these project records.

The evidence covers historical reuse, insertion-position diagnosis, frozen-ranker validation, and complete-GP position evaluation. The historical study's aggregate directory contains 120 run summaries and 3,690 instance results. The position study's combined aggregate directory contains 90 run summaries and 2,790 instance results. Local parent-fragment groups and complete-search seeds retain their respective analysis units.

The position analysis uses 30 distinct seeds from fragment_position_gp_equal_time_01 and fragment_position_gp_equal_time_20seed_extension_01. Combined tables are stored in the latter directory's aggregate folder. Completion counts use seed-weighted averages, and sortable proposals use totals. Paired bootstrap calculations use the saved per-instance results and the archived analysis configuration.

Table A1. Archived Materials for Reproduction

Artifact	Archived contents
Data and run manifest	Generator, floor/round rules, development and test instances, all seed IDs, method labels, budgets, hashes.
Grammar and archive	Typed operators, constants, fragment ASTs, provenance, exclusion normalization, and duplicate rules.
Ranker and probes	Frozen feature order, scales, coefficients, tie rules, probe states and hashes, with source-manifest references.
Execution environment	Hardware, workers, dependencies, code revision, historical LLM version, quantization and sampling parameters.
Search and timing traces	All proposed sites, selected positions, failures, duplicates, complete evaluations, population updates, cache policy, legal submission and cleanup times.
Final outputs	Frozen programs, per-instance bins, paired seed results, and cost records.

The repository supplies the shared core in fragment_gp_limited_01, both position-study source folders, the frozen ranking models and probes, data, final programs, and complete search traces. Large trace collections are stored in ZIP archives with per-file hashes. The guide docs/POSITION_GP.md documents verification, trace restoration, final-program evaluation, and preparation of new reproduction runs. The package manifest verifies the distributed files.